SOURCE REVIEW AND CHECKED REFACTORINGS

Shortening and simplifying the DiophantineConsistency Lean proofs

Prepared for Vladimir Reshetnikov

26 October 2026

Repository	https://github.com/VladimirReshetnikov/ProveIt
Reviewed commit	60e9c5d620c6c387d932876cfa2d9376f64ea56c
Lean	leanprover/lean4:v4.32.0
Mathlib	81a5d257c8e410db227a6665ed08f64fea08e997
Scope	Computability/DiophantineConsistency

Main recommendation. Concentrate on repeated proof structure and missing shared interfaces: uniform coefficient bounds, library lookup facts, successful decoder cases, assignment transport, finite residual sums, and scanner invariants. Preserve the executable expressions, public statements, and each theorem's axiom dependencies while reducing the source burden.

Contents

1 Results and priorities **3**

1.1 What the survey covers . . . 3

2 Replace repeated coefficient bounds with one argument **4**

2.1 The existing pattern . . . 4

2.2 The replacement . . . 4

2.3 Proof budget and maintenance . . . 5

2.4 A certificate design that could remove repeated arithmetic search . . . 5

3 Use existing lookup, sum, and arithmetic lemmas **6**

3.1 Seven list inductions have direct replacements . . . 6

3.2 A finite sum at most one . . . 6

3.3 Natural subtraction and remainder . . . 6

3.4 Let `omega` handle the cast interface it already supports . . . 7

3.5 Keep huge powers symbolic . . . 7

4 Shorten decoder proofs without enlarging their axiom dependencies **7**

4.1 Repeated successful-decode cases . . . 7

4.2 Why the apparently shorter fusion is not the default patch . . . 8

5 A small assignment-transport API removes repeated extensionality **8**

5.1 The invariant already implicit in the constructors . . . 8

5.2 Data preservation is checked, not inferred from equivalence . . . 9

6 Share the finite residual and degree arguments **9**

6.1 Three copies of the same evaluator lemma . . . 9

6.2 The needed polynomial-degree theorem already exists . . . 10

6.3 Regroup witnesses with the existing sum-type theorem . . . 10

6.4 Two further consolidations . . . 10

7 Reuse scanner and validated-history proofs **11**

7.1 Recursive steps and terminal states . . . 11

7.2 An input row and a stored judgment need not have the same type . . . 11

8 Simplify object-theory endpoints at the semantic boundary **11**

8.1 A completeness-and-sealing helper . . . 11

8.2 Reuse the ZFC-model-to-ZF-axioms bridge . . . 12
8.3 A smaller quantified-list example . . . 12

9 Unify modular and finite-word interfaces 12
9.1 Share residue transport at the correct modulus . . . 12
9.2 One boundary between `ZMod`, `Fin`, and lists . . . 12
9.3 Package repeated geometric facts without adding assumptions . . . 13

10 Treat generated circuit proofs as generated interfaces 13
10.1 What the large files actually contain . . . 13
10.2 Reuse the existing power-certificate theorem . . . 13
10.3 A naming trap worth avoiding . . . 14

11 A reusable invariant for arithmetic DAG evaluation 14

12 Validation and integration plan 14
12.1 What has been checked . . . 14
12.2 Replay the evidence . . . 15
12.3 Suggested order . . . 15

13 Further questions 16

A Source and artifact guide 16

1 Results and priorities

The strongest opportunities are concrete and local. Several long passages repeat the same mathematical argument with different indices or repeat facts already available in Lean, Mathlib, or a neighboring ProveIt module. The proposed changes therefore start by making those facts reusable. A wholesale change of the mathematical encoding would add considerably more verification work than these proof refactorings require.

This package separates three kinds of evidence. A *full-file replay* checks a replacement file with its original imports and declarations. A *checked prototype* establishes a proposed reusable lemma or a copied original theorem statement, without claiming that every client module has been migrated. An *architectural proposal* identifies a source-specific change that still needs implementation and measurement.

Change	Concrete benefit	Evidence in this package
Uniform row bounds	Replace 22 repeated coefficient proofs and their dispatch with one finite-case proof.	Full replacement file: 679 to 122 lines; slower in the recorded elaboration run.
Core list lookup lemmas	Seven unchanged declarations across five files: 61 to 21 physical lines.	Core wrappers and a full ProofChecker replay.
Decoder success cases	Replace nested `none`/`some` trees with two uses of the successful-bind characterization.	Full replacement file; unchanged per-theorem axiom lists.
Assignment transport	Three representation constructors share two lemmas; 260 to 242 lines, including the helpers.	Full replacement file and unchanged-expression checks.
Finite residual API	Three compilers share zero-sum and degree arguments; two also share witness regrouping.	Eight exact replacements; four patched modules and their selected audits replayed.
Scanner and history APIs	Share recursive-step, terminal-state, append, and invariant arguments.	Generic lemmas and specialization bridges checked.
Small arithmetic changes	Reuse finite-sum, division/remainder, power, and cast results with the same hypotheses.	Checked exact-statement examples.
Object semantics and generated circuits	Reduce completeness boilerplate, reuse degree interfaces, and certify power tables.	Source-specific proposals; integration remains future work.

The source-line measurements are deliberately narrow. They include helper definitions when a helper is part of a proposed patch. They are not estimates of a repository-wide percentage reduction, proof-term size, compilation speed, or the size of the resulting Diophantine equation. The equation data remain unchanged in the accepted local refactorings.

1.1 What the survey covers

The pinned subtree contains 711 tracked Lean files. The maintained `Lean` tree contains 546 files; 363 of them lie directly in the `Lean/DiophantineConsistency` module directory, totaling 68,308 physical lines. That directory includes 38 files with a `Checks` or `Audit` suffix. Counting the separate `Lean/checks`

directory gives 220 audit/check files in the maintained tree. The remaining tracked Lean files in the requested subtree are in historical proposal/provenance and candidate directories. They should not be counted as additional live implementations to refactor automatically.

The complete filename, byte count, line count, and SHA-256 inventory is in `evidence/source-inventory.csv` and its JSON counterpart. These numbers come from the supplied `scripts/inventory.py`. Its tactic and duplicate-name counts are textual indicators; they are not Lean AST analysis. In the main module directory, after removing comments and strings, the survey found no `sorry`, `admit`, `axiom`, or `native_decide` tokens. This is not a claim that imported theorems have no axioms. The separate kernel dependency checks matter precisely because a textual scan cannot establish that.

2 Replace repeated coefficient bounds with one argument

2.1 The existing pattern

NativeCoefficientBounds:L63 begins a sequence of 22 private `rowEnvelope_i` lemmas. Each lemma fixes one row, unfolds its literal coefficient recipe, and proves that every branch of a nested conditional is bounded by the same envelope. The public dispatcher starts at NativeCoefficientBounds:L602. The repeated block contains 566 physical lines, including spacing and its explanatory text.

For ports $p_0, p_1, p_2, p_3 \geq 0$, the common envelope is

$$E = 2p_0^2 + 2p_3p_0 + p_2 + p_3 + 2p_0 + p_1p_0 + p_1 + 8.$$

Every branch proof uses exactly the same facts: the four nonnegativity hypotheses, $p_0^2 \geq 0$, $p_3p_0 \geq 0$, and $p_1p_0 \geq 0$. Once these products are treated as known nonnegative terms, the final obligations are linear arithmetic. There is no need to copy the entire row expression into a separate `heq : ... := rfl` in every lemma.

2.2 The replacement

The important operation is to apply the existing conditional-bound lemma uniformly, rather than split on every equality test and accumulate unused branch hypotheses. The core of the candidate is:

```
have hsq := sq_nonneg (ports 0)
have hmf := mul_nonneg h3 h0
have hkp := mul_nonneg h1 h0
fin_cases k <;> dsimp only [rowCoefficient]
all_goals
  unfold envelope
  repeat' apply abs_ite_le
  all_goals
    rw [abs_le]
    constructor <;> linarith only [h0, h1, h2, h3, hsq, hmf, hkp]
```

Here `abs_ite_le` is the existing private helper: if both branch values have absolute value at most E, then so does the conditional. The proof is mathematically identical to the original strategy. The shorter script states that strategy once and lets it act on all 22 literal rows.

The full replacement file compiles against the original imports. Excluding the review-only axiom-print commands, it shrinks from 679 to 122 physical lines, saving 557 lines (about 82%). The repeated row block shrinks from 566 to 15 lines, including a local `maxHeartbeats` option; a separate symbolic-power simplification accounts for the remainder. The three audited exported theorems retain their original standard axiom lists. The coefficient table, envelope definition, and all public theorem signatures are unchanged. The patch is supplied as an optional source-maintenance tradeoff in `patches/optional/coefficient-bounds-compact.patch`.

The public assumptions, including all four nonnegativity hypotheses, are essential to this argument and remain present. The patch does not assert that the bound holds for arbitrary signed ports, and it does not alter the coefficient table or the stated published height.

2.3 Proof budget and maintenance

Combining these small proofs into one does change the heartbeat budget: the default 200,000 limit timed out. The checked candidate uses a local `set_option maxHeartbeats 4000000 in` on the combined theorem. This makes its proof budget explicit. It should not become an unexplained file-wide or project-wide increase. A proposed alternative retaining separate row declarations was also tested, but hit the default heartbeat/simplifier limits on a large row. It is not included among the checked replacement files. A future shared branch-proof recipe or checked coefficient certificate is preferable to pretending that this fallback is already validated.

Source reduction alone does not show smaller proof terms or faster elaboration. In one sequential comparison using one Lean thread, the original file used 23.37 seconds of user CPU time and the compact file used 110.40 seconds; wall times were 24.42 and 111.37 seconds. Both runs exited successfully under the same 3,000 MB Lean memory cap. These are single-run diagnostics on a shared host, not a general benchmark, but the observed roughly 4.7-fold CPU increase is enough to reject an unqualified performance improvement claim. Prioritize the other checked patches when build time matters; use this compact version when the reduced proof-maintenance burden is worth that tradeoff.

2.4 A certificate design that could remove repeated arithmetic search

The common envelope is a nonnegative linear combination of a fixed eight-entry basis:

$$v = (p_0^2,\ p_3p_0,\ p_2,\ p_3,\ p_0,\ p_1p_0,\ p_1,\ 1), \qquad b = (2,2,1,1,2,1,1,8).$$

For each literal branch, record an integer coefficient vector q with $|q_i| \le b_i$. A single triangle-inequality theorem gives

$$\left|\sum_i q_i v_i\right| \le \sum_i |q_i| v_i \le \sum_i b_i v_i = E.$$

A generator could emit the small vectors, ordinary decidable componentwise bound certificates, and kernel-checked equalities connecting their evaluations to the actual row expressions. This would remove repeated linear-arithmetic search, rather than merely its repeated source spelling. The exact binding to the existing coefficient definition remains an obligation. This certificate architecture is a proposal, not an implemented or measured improvement in this package.

3 Use existing lookup, sum, and arithmetic lemmas

3.1 Seven list inductions have direct replacements

Five modules independently prove that a successful optional list lookup returns a member of the list. Two also prove the converse existence of an index for a member. The exact core lemmas are `List.mem_of_getElem?` and `List.getElem?_of_mem`, from the pinned Lean release's `Init/Data/List/Lemmas.lean`.

```
theorem lookup_mem {α : Type} {xs : List α} {n : Nat} {x : α}
    (h : xs[n]? = some x) : x ∈ xs :=
  List.mem_of_getElem? h

theorem exists_lookup {α : Type} {xs : List α} {x : α}
    (h : x ∈ xs) : ∃ n, xs[n]? = some x :=
  List.getElem?_of_mem h
```

The patch preserves the original local declaration names and theorem statements. It changes only their proof bodies. Its seven complete declarations go from 61 to 21 physical lines. The sites are `AnnotatedAssumption`:L40, `AnnotatedRuleSide`:L18, `AnnotatedBound`:L50, `AnnotatedRename`:L110, and `AnnotatedProofChecker`:L177. The last file was replayed in full, with axiom checks on the changed lemma and on its soundness/checker endpoints. The other four clients are supported by the exact core-lemma prototypes and a clean patch application check; their entire dependency closures have not been rebuilt for this package.

This is a good first integration change: it removes bespoke inductions without creating a new project helper, import layer, or computational definition.

3.2 A finite sum at most one

`CurrentCellLabels2011`:L131 proves that a natural-valued finite function whose sum is at most one is either identically zero or has one entry equal to one and all other entries zero. Mathlib already characterizes this situation with `Finset.sum_le_one_iff`. The nearby module `NativeSelectorClause54`:L65 already uses that lemma.

The checked replacement reduces the original 27-line proof body to 10 lines. It uses the library characterization, splits the zero case from a nonzero entry, and reconstructs the exact original conclusion. The type remains an arbitrary `Fintype`; the argument also handles an empty type. This is a particularly useful style of simplification: transfer an established local method to another development, instead of adding a parallel lemma with a slightly different statement.

3.3 Natural subtraction and remainder

In `CurrentCellTranslation2011`:L17, the whole-cell portion of a shift is `t - t % 2011`. Two short facts can be stated directly using core division/remainder identities. In particular,

```
theorem wholeShift_add_remainder (t : Nat) :
    (t - t % 2011) + t % 2011 = t :=
  Nat.sub_add_cancel (Nat.mod_le t 2011)
```

The inequality is still explicit in the proof term. The argument does not treat truncated natural subtraction as subtraction in a group. The companion divisibility example uses `Nat.mul_div_self_eq_mod_sub_self` to supply the quotient witness directly. Both examples compile with only the standard library.

3.4 Let `omega` handle the cast interface it already supports

NativePellStepDown:L161 contains `eq_of_residues`, whose final reasoning is Presburger arithmetic relating natural bounds and integer casts. The checked small example removes four manually introduced cast facts and lets `omega` use the original hypotheses directly. The complete original and replacement files both compile with matching audit output for the changed theorem and two source-index endpoints. The replacement retains every statement and reduces the file from 248 to 241 physical lines, excluding appended audits. An initially imposed 3,000 MB Lean cap stopped both files during import; both passed when replayed sequentially with a 6,000 MB cap. This was an environment limit shared by the original and replacement, not a failed arithmetic argument.

This is a targeted use of automation. It is not a recommendation to replace nonlinear Pell arguments with an unstructured `omega` or `nlinarith` call. It works at a specific interface where the arithmetic is already linear and the cast relations are supported.

3.5 Keep huge powers symbolic

The published-scale lemmas in NativeCoefficientBounds:L647 already avoid expanding million-bit integers. That is the right design. The `published_large_scale` proof can be shortened further by expressing 16 as 2^4, combining powers with `pow_add`, and invoking `pow_le_pow_right₀` for the exponent comparison. This preserves the symbolic proof strategy; it does not use a giant numerical normalization.

4 Shorten decoder proofs without enlarging their axiom dependencies

4.1 Repeated successful-decode cases

AnnotatedProofCode:L78 and AnnotatedProofCode:L107 prove two properties of a successful fuel-bounded tree decoder:

$$\text{decode}(f, c) = \text{some}(t) \quad \Longrightarrow \quad \text{depth}(t) \le f, \qquad \text{quote}(t) = c.$$

The inductions are over the same fuel parameter. In the fork case, both proofs explicitly split the left recursive decode into `none` and `some`, then do the same for the right decode. The unsuccessful branches contradict the successful outer result.

The useful abstraction is the successful-bind characterization for `Option`. After reducing the decoder branch, apply `Option.bind_eq_some_iff.mp` directly to the hypothesis. Obtain the left decoded tree

and its equation, then obtain the right decoded tree and its equation. The final `some` equality identifies the output tree. This removes the impossible branches from the source proof.

Direct application is significant here: a `simp` rewrite with the same theorem did not match one already-reduced bind shape in the tested development. The supplied file records the working elaborated version.

Both the isolated original decoder definitions and the complete modified `AnnotatedProofCode` file compile. The baseline and replacement audit output is byte-for-byte identical for the two directly changed theorems and both checked checker endpoints. The recommended patch is `patches/decoder-options.patch`.

4.2 Why the apparently shorter fusion is not the default patch

A combined theorem asserting both conclusions shortens the original 55-line block to 35 lines, with the old names as projections. That version does compile. However, the original depth-bound theorem depends on `propext` and `Quot.sound`; the quotation theorem additionally depends on `Classical.choice`, inherited through `Nat.pair_unpair`. Projecting the depth bound from a proof of the conjunction makes the depth theorem inherit that extra axiom too.

The accepted approach keeps the inductions independent and compresses their `Option` reasoning. Its block is 43 lines, saving 12, while retaining the original dependency lists:

Theorem	Original and accepted replacement axioms
`decoded_depth_le`	`propext`, `Quot.sound`
`quote_of_decode`	`propext`, `Classical.choice`, `Quot.sound`

The final checker endpoints already used the standard classical axioms, so comparing only those endpoints would have missed the local change. This example gives a practical reason to audit the directly refactored theorems as well as their main consumers. The fused prototype is included as an explicitly non-default alternative, with its tradeoff recorded.

5 A small assignment-transport API removes repeated extensionality

5.1 The invariant already implicit in the constructors

`Representation:L155` defines `Rep.reindex`, followed by conjunction and disjunction constructors. They preserve executable syntax and package correctness proofs with an explicit number of witnesses. Several local proofs establish the same identity for the two components of a sum-indexed assignment.

For input assignments v, witness assignments w, and renamings f, g, the common fact is

$$\mathrm{natAssignment}(v, w) \circ \mathrm{Sum.map}(f, g) = \mathrm{natAssignment}(v \circ f, w \circ g).$$

Its proof is function extensionality followed by the two constructors of `Sum`. Combining it with the existing `Expr.eval_rename` gives the evaluator-level interface:

```
theorem eval_rename_natAssignment (e : Expr (σ ⊕ α))
    (f : σ → τ) (g : α → β) (v : τ → Nat) (w : β → Nat) :
    (e.rename (Sum.map f g)).eval (natAssignment v w) =
      e.eval (natAssignment (v ∘ f) (w ∘ g)) := by
  rw [Expr.eval_rename, natAssignment_map]
```

The correctness field of reindexing then becomes:

```
correct v := by
  simpa only [eval_rename_natAssignment, Function.comp_id]
    using r.correct (v ∘ f)
```

Conjunction and disjunction use the same lemma for the left and right blocks of a finite witness vector. Their remaining proof is the actual logical work: combine or separate the witnesses, then use the zero-sum-of- squares or zero-product characterization.

5.2 Data preservation is checked, not inferred from equivalence

The complete replacement file compiles. Its size changes from 260 to 242 physical lines, including 15 new helper lines. The separate prototype also checks by `rfl` that all three new constructors have exactly the same `expression` fields as the original constructors. Their witness counts are unchanged by construction.

This distinction matters in this repository. A shorter proof of a merely equivalent solvability proposition would not by itself justify claims about the same executable equation or the same witness count. Here the data fields are preserved, and only the supporting correctness proofs are reorganized. Keep these helpers near `natAssignment` in the representation layer so that later compilers can reuse them without introducing reverse imports.

6 Share the finite residual and degree arguments

6.1 Three copies of the same evaluator lemma

The private `eval_sumExpressions` induction appears in `DAGQuartic:L83`, `NaturalDAGQuartic:L110`, and `QuadraticRegisterSystem:L70`. Each client then proves the same zero-sum-of-squares characterization for its residual expressions.

The shared interface should take arbitrary finite residual syntax $e_i : \mathrm{Expr}(\sigma)$ and an integer assignment v:

$$\mathrm{eval}\left(\sum_{i:\mathrm{Fin}\, n} e_i^2, v\right) = 0 \quad \Longleftrightarrow \quad \forall i,\ \mathrm{eval}(e_i, v) = 0.$$

Make the evaluation lemma reusable, then prove this characterization once. Each of the three client theorems becomes an instantiation. The $n = 0$ case is covered by the finite statement: the empty sum is zero and the universal condition is vacuous.

Keep the domain assumptions honest. The proposed lemma is about integer evaluation. The corresponding assertion is false in an arbitrary ring; for example, nonzero complex squares can cancel. A more general theorem would need explicit ordered-ring or suitable sum-of-squares hypotheses.

6.2 The needed polynomial-degree theorem already exists

For residual degrees at most d, the compiled sum of squares has total degree at most $2d$. The H10 common library already supplies `Diophantine.totalDegree_sum_pow_le`. The proposed shared helper uses it rather than proving another polynomial-degree induction. The three degree clients are `DAGQuartic`:L244, `NaturalDAGQuartic`:L253, and `QuadraticRegisterSystem`:L150.

This is an upper bound, not an assertion that the degree is exactly $2d$. Cancellations or zero residuals can lower the degree. The proposed API retains the original inequalities.

6.3 Regroup witnesses with the existing sum-type theorem

The proofs at `DAGQuartic`:L260 and `NaturalDAGQuartic`:L267 manually split an assignment on a sum type into its two components, reconstruct it with `Sum.elim`, and prove cast identities by extensionality. Mathlib's `Sum.exists_sum` already gives the existential regrouping. A single shared cast identity then allows both clients to use their existing `exists_zero_iff` theorem through `exists_congr`.

The topological, allowed-node, and output-index hypotheses are retained. The shorter proof does not assume the evaluation/constraint correspondence; it invokes the existing theorem that establishes it.

The checked patch adds a 36-line `CompilerSupport` module and changes the three client files from 285 to 257, 293 to 264, and 170 to 158 physical lines. The net saving, including the new module, is 33 physical lines (36 nonblank lines). All eight original/replacement theorem pairs have the same audited standard axioms. An isolated overlay also compiled all four patched modules, eight unchanged dependencies needed by the audits, and all three existing audit files: 15 successful commands. The overlay does not modify the original checkout. It is a targeted integration replay, not a complete downstream rebuild.

6.4 Two further consolidations

Generic sum cost. The private `DAGQuartic.sumExpressions_cost` is general in a per-expression bound b. `QuarticCost.sumExpressions_binaryOps_le` separately proves the case $b = 5$. Move the generic statement into the common cost module, keep the old specialized name as a compatibility wrapper, and instantiate the signed DAG's bound at $b = 11$. Directly substituting the existing $b = 5$ theorem for a $b = 11$ use would be incorrect.

Weighted syntax degree. `QuadraticRegisterSystem.syntaxDegree` is the constant-weight-one case of the existing `Expr.weightedDegree` in `WeightedExpressionDegree`:L1. Its polynomial-degree proof can be derived from `degree_evalRing_le` and `evalRing_X`, after proving an API-preserving equality for the syntax measure. Check the import graph before moving this fact; a small common degree module is preferable to a cycle between compiler clients.

These last two changes are proposals in this package. They should be implemented after the smaller checked interfaces, not combined with an unrelated redesign of the expression datatype.

7 Reuse scanner and validated-history proofs

7.1 Recursive steps and terminal states

The annotated scanner modules repeat proofs about a `Nat.rec` iteration: its zero case, its successor equation, and the fact that a finished or rejected state stays fixed. Relevant families include `AnnotatedBoundScan`, `AnnotatedLineScan`, `AnnotatedRenameScan`, `AnnotatedAxiomContext`, and `AnnotatedContext`. Representative anchors are `AnnotatedBoundScan:L30`, `AnnotatedLineScan:L35`, `AnnotatedRenameScan:L34`, `AnnotatedAxiomContext:L32`, and `AnnotatedContext:L100`.

The checked `lean/ScanIteration.lean` prototype proves generic recursive-step and fixed-state facts and checks concrete scanner shapes. One can reuse these proof lemmas while leaving every executable scanner definition intact. That is a useful first step because scanner arithmetic, primitive-recursive encodings, and size theorems may depend on the literal definition shape.

7.2 An input row and a stored judgment need not have the same type

`AnnotatedProofChecker`, `AnnotatedRename`, and `AnnotatedBound` repeat an append theorem, soundness induction, and `Produces.append`/`Produces.step` arguments for a validator that pushes a new item onto a history. The reusable runner takes:

- an input type I and a stored-history type S;
- a validator $I \to \mathrm{List}(S) \to \mathrm{Bool}$;
- an output map $I \to S$;
- a local soundness theorem for a predicate on stored items.

The generic invariant states that a successful run preserves that predicate throughout the history. The output map is necessary: a proof-checker line may store a judgment rather than the entire input line. A shared runner with a single type would not cover all three clients accurately.

The prototype `lean/ValidatedHistory.lean` checks the generic runner, all four reusable theorems, and bridges from unchanged concrete recursion shapes. Those bridges are proved function equalities; the concrete recursive definitions are not automatically definitionally equal to a specialization of a separately defined generic runner. Replacing them by an unjustified `rfl` would fail.

For integration, first keep the concrete definitions and use the proved bridges in theorem wrappers. Only consider redefining the executable runner itself after checking the downstream numeric and primitive-recursive representations. The generic prototypes are checked; migration of every concrete client remains future work.

8 Simplify object-theory endpoints at the semantic boundary

8.1 A completeness-and-sealing helper

Many `Object*` endpoints follow the same sequence: invoke first-order completeness, introduce an arbitrary model and its ZFC-model hypothesis, remove the empty assumption context, rewrite the sealed formula's semantics, and apply a previously established semantic lemma. The survey found

27 completeness invocations in these files, with two notably different shapes. About 25 are plausible clients for a helper that turns validity of the unsealed body over arbitrary models into a ZFC derivation of its seal. One representative endpoint is `ObjectNaturalExpression:L182`.

The candidate helper belongs in a low common bridge layer. It must quantify over the same arbitrary models, environments, and formula semantics as the existing completeness theorem. It should encapsulate only the repeated completeness/sealing steps. The arithmetic content remains the semantic theorem supplied by the caller.

This is not permission to replace the object model's natural-number domain by the metatheory's `Nat`. The development must still handle its full internal ω, including nonstandard behavior allowed by the model semantics. The existing generic semiring interface in `ObjectNaturalSemiring.evalRel_iff` is the appropriate kind of abstraction downstream. Using it in its own foundational dependency `ObjectNaturalExpression` would create an import cycle in the current organization unless the foundational definitions were split deliberately.

8.2 Reuse the ZFC-model-to-ZF-axioms bridge

Two copies of the six-axiom extraction are `ObjectArithmetic:L143` and `ObjectMultiplication:L169`. There is also an existing theorem `LeanProofs.BoundedZFCConsistency.zfAxioms_of_zfcModel` in `CodedRank`. Extract a suitable common theorem or expose a low-layer alias. Importing the large later `CodedRank` development into an earlier arithmetic foundation simply to save these lines would be a poor dependency tradeoff.

8.3 A smaller quantified-list example

`ObjectNaturalExpression.inputsNaturalForm_spec` manually constructs and destructs membership in `List.ofFn`. Mathlib already has `List.forall_mem_ofFn_iff`. The anticipated proof is a short `simp only` chain through the conjunction semantics, that list theorem, and `fNatF_spec`. This is a precise next candidate, but its complete original-import replay is not included here; treat it as a proposal rather than a checked patch.

9 Unify modular and finite-word interfaces

9.1 Share residue transport at the correct modulus

`CurrentClause2011:L27` has public addition/subtraction residue theorems whose proof bodies are duplicated by private helpers in `CurrentReflectedGeometry2011:L74`. Put the common facts in a lightweight cyclic-residue module imported by both clients. A checked generalized prototype appears in `lean/SmallArithmetic.lean`.

The modulus must remain the full cyclic modulus used in the application, such as NL. Replacing it everywhere by the local cell length L loses whole-cell position information. A small lemma transporting equality of residues is safe; silently quotienting away a whole-cell offset is a change of theorem.

9.2 One boundary between `ZMod`, `Fin`, and lists

`NativeCyclicConvolution:L1` and `NativeRadixDigits:L1` repeatedly move between cyclic functions,

finite functions, padded digit lists, and finite sums. A common API should contain the evaluation of a word built from `List.ofFn`, the lookup identity through the `Fin`/ `ZMod` equivalence, and the finite-sum transport theorem. Then word reconstruction and mask statements can be proved at the same abstraction level before crossing that boundary once.

The existing `word_eq_ofDigits`, `ofDigits_ofFn`, and `finEquiv_val` lemmas already do part of this work. The proposal is to provide the missing composition as a shared interface, not introduce a second digit representation. Preserve the `NeZero` modulus instance, digit-capacity bounds, and padded-list length hypotheses at the boundary.

9.3 Package repeated geometric facts without adding assumptions

The current-cell and reflected-geometry proofs repeatedly derive positivity of a modulus, source-index bounds, and wrapped/nonwrapped index equations. A small record of *proved consequences* can shorten subsequent calculations. Its constructor should derive those fields from the existing parameters; it must not turn an unproved global alignment or label-binding fact into an implicit typeclass assumption. Keep these records explicit arguments when two different geometric situations may coexist.

This remains a design proposal. The immediate residue and natural-remainder examples are much lower-cost starting points.

10 Treat generated circuit proofs as generated interfaces

10.1 What the large files actually contain

The two generated step files dominate the raw line-count list:

File	Physical lines	Step lemmas	Power lemmas
`GapCircuitSteps66`	3,122	484	86
`SplitCircuitSteps59`	3,445	515	107
Total	6,567	999	193

Each step lemma already instantiates the shared `run_node_equation`. These are not 999 distinct missing mathematical arguments. A source-volume improvement would change the generator and its public interface: expose one indexed theorem, then keep numerical aliases only where clients or audit scripts need stable names.

Do not hand-edit generated output and leave the generator unchanged. The report's proposal targets the step generators as well as their output, with a regeneration check in the integration gate.

10.2 Reuse the existing power-certificate theorem

The 193 power lemmas contain 193 local `ring` invocations. The existing `ArithmeticDAG.run_power` in `ArithmeticDAGPowers:L24` already proves correctness of exponent labels under `PowerStep` conditions. The generators already compute the relevant exponent information. A useful next design is:

1. generate one exponent-label table per circuit;

2. certify its local `PowerStep` obligations in Lean;

3. invoke `run_power` once;

4. recover any required `natural_power_*` names as short aliases.

This is a potential reduction in repeated normalization, but it is not implemented or benchmarked in this package. The size and elaboration cost of the checked table must be measured against the original 193 proofs. Use ordinary kernel-checked certificates; substituting `native_decide` would change the verification mechanism and is not an automatic source simplification.

10.3 A naming trap worth avoiding

The names `naturalNodes` in the Gap/Split developments describe natural inputs. They do not imply that every intermediate node satisfies `NaturalDAGQuartic.Allowed`. The Gap list contains subtraction nodes and negative literals. A shared natural-semiring proof must not be applied solely because the identifier contains "natural." Retain the explicit allowed-node certificate wherever that theorem requires it.

11 A reusable invariant for arithmetic DAG evaluation

Two existing proofs use nearly the same strong induction along earlier dependencies: `NaturalDAGQuartic.run_nonnegative` and `ArithmeticDAG.value_degree_le`. A generic `run_invariant` can take a predicate $P(i, v)$ and a local closure rule for each node given the invariant on its dependencies. Nonnegativity and degree bounds then become local closure proofs.

The repository already has `run_node_equation` and `run_eq_of_node_equations`; those theorems are the natural basis for such an interface. Do not duplicate their semantic correspondence in a new evaluator. A small common induction lemma is sufficient.

The `lean/RunInvariant.lean` prototype compiles, including the shared induction rule and both exact application statements. The two original/replacement application pairs have matching axiom lists; all directly printed lists contain only the standard axioms. This establishes the reusable proof interface; moving it to its final import layer and replacing the original two proof bodies remain an integration step.

A more ambitious alternative would make the natural-register evaluator semiring-polymorphic and relate it to the erased integer DAG by an explicit erasure-correctness theorem. This could reduce cast and nonnegativity proofs, but would also introduce new dependent-index transports. It is a later research direction, not the recommended first refactoring.

12 Validation and integration plan

12.1 What has been checked

The evidence files record the exact compiler version, exit status, source hashes where supplied, and axiom output for the tested examples. A successful audit requires compiler exit status zero and no proof errors before its axiom output is considered. In one timed-out trial, an axiom-printing command still emitted an innocuous-looking line; that line did not make the failed declaration a valid proof.

All accepted checked examples use ordinary Lean proofs. The local source refactorings do not add an admitted theorem or a new axiom. The rejected default decoder fusion is retained separately so that its verified *increase in the local axiom dependency list* is visible.

The completed checks include the two syntax replacements with paired full-file axiom logs; the assignment patch and its executable-data equality checks; the finite-residual patch with its 15-command overlay replay; the DAG invariant prototype; the full original/compact coefficient files; the full original/shortened Pell-step files; and the small core/arithmetic prototypes. Across the structural examples, 13 original/replacement theorem pairs have equal audited axiom sets. The `verification-plan.json` file and topic receipts give the exact scope for each item.

The environment was shared and resource-constrained. Timings are single-run diagnostics rather than reliable general performance estimates. The observed coefficient-proof slowdown is reported explicitly because it affects the integration recommendation; no speedup is claimed. The initial broad prerequisite build was replaced by a sequential build of the needed missing modules. The compiler inputs remained the pinned Lean release, the pinned Mathlib revision, and the original ProveIt sources. Toolchain/cache preparation is not included in the ZIP.

12.2 Replay the evidence

`scripts/verify.py` is the entry point for replaying the packaged examples against a checkout at the pinned commit. The accompanying `README.txt` specifies prerequisites, the available verification levels, and which artifacts are proposals. The script checks the recorded source snapshot and compiles the requested files; it does not fetch a different Mathlib revision or silently accept a newer repository commit.

The report and patches are designed for review and integration. A full build of all 546 maintained Lean files, or every downstream consumer of the combined patch, is outside the validation claim of this package. The next integration gate should replay affected import dependents, especially where a private helper is removed, a shared lemma is introduced, or a simplification attribute changes.

12.3 Suggested order

1. Apply the list-lookup and small arithmetic replacements. Keep public names and statements unchanged and compare the directly changed theorem axioms.
2. Apply the independent decoder simplification and assignment-transport patch. Retain the expression-data equality checks for the latter.
3. Review the optional compact coefficient-bound proof separately. Its local heartbeat budget and observed elaboration cost are material tradeoffs, even though it passes the original published-bound statements.
4. Introduce the finite-residual support module and migrate its three clients together. Count the new module when measuring the reduction.
5. Introduce scanner/history proof interfaces while preserving concrete definitions through explicit bridges.
6. Review the object-sealing and generated-power designs separately, because their import/API and regeneration consequences are broader.

For each change, distinguish four properties: the public proposition is unchanged; executable data are unchanged where relevant; the directly changed theorem's axiom list has not grown; and affected dependents compile. One property does not automatically imply the others.

13 Further questions

1. Can a small tool flag repeated extensionality and recursive-case proof shapes, leaving the choice of abstraction to a maintainer?
2. Can the eight-term coefficient-basis certificate remove the measured elaboration overhead of the compact row-bound proof while retaining its short source and exact literal binding?
3. Does a `PowerStep` label-table certificate reduce total elaboration work or serialized proof size relative to the 193 generated power proofs? Measure both metrics; a shorter generator alone does not answer the question.
4. How small can the common residual API remain while serving the signed DAG, natural DAG, and quadratic-register compiler? A good boundary should avoid importing a full compiler merely to use a finite-sum lemma.
5. Can a successful-decode induction principle reduce both decoder proofs while preserving their separate constructive/classical dependency footprints? The conjunction experiment shows exactly what must be avoided.
6. How much object-level boilerplate disappears after one carefully placed completeness-and-sealing theorem, while preserving all quantification over arbitrary models and internal ω?

A Source and artifact guide

All source links in this report point to the pinned repository revision. Core-list references are to Lean's list lemmas at the release commit. Mathlib reuse is grounded in the pinned source, including `Sum.exists_sum`, `Finset.sum_le_one_iff`, and `List.forall_mem_ofFn_iff`.

Artifact	Purpose
`*.tex`, `*.pdf`	This complete report, with editable LaTeX source.
`patches/`	Independent reviewable patches; patch ordering and status are described in the README.
`lean/`	Checked prototypes and full replacement files, with clear distinctions between accepted candidates and optional experiments.
`evidence/`	Source inventory, patch metrics, validation receipts, axiom logs, and a precise status manifest.
`scripts/inventory.py`	Reproduce the maintained-source inventory.
`scripts/verify.py`	Replay selected examples and replacement-file checks.
`README.txt`	Contents, prerequisites, replay instructions, integration cautions, and status definitions.
`SHA256SUMS.txt`	Integrity hashes of the packaged files.